Highly Available, Auto-Scaled Web Application on AWS

Design, Deployment, and Hardening of a Multi-Tier Architecture in a Custom VPC

Mosab Kamal
Zewail City of Science and Technology

Abstract

This report documents the design and deployment of a highly available, auto-scaled web application on Amazon Web Services. The solution is built inside a custom VPC that spans two Availability Zones, with web servers placed in private subnets, public traffic served through an Application Load Balancer, and outbound access provided by one NAT Gateway per Availability Zone. Instance management uses AWS Systems Manager (SSM) instead of SSH keys, and the security groups are restricted following the principle of least privilege. The report covers the architecture, implementation steps, verification, security decisions, and cleanup.

Contents

1 Overview **2**
1.1 Objective . . . 2
1.2 Technologies Used . . . 2

2 Architecture **2**
2.1 Design Summary . . . 2
2.2 Network Layout . . . 3
2.3 Routing . . . 3

3 Implementation **3**
3.1 Step 1: VPC and Subnets . . . 3
3.2 Step 2: NAT Gateways . . . 3
3.3 Step 3: IAM Role for Systems Manager . . . 4
3.4 Step 4: Web Servers . . . 4
3.5 Step 5: Application Load Balancer . . . 4
3.6 Step 6: Launch Template and Auto Scaling Group . . . 4
3.7 Step 7: Security Hardening . . . 5

4 Verification **5**

5 Design Decisions **5**

6 Cleanup and Cost **6**

7 Lessons Learned and Future Work **6**

1 Overview

1.1 Objective

The goal of the project was to build a web tier that stays available if a whole Availability Zone fails, scales with demand, and is not directly reachable from the internet. Only the load balancer is exposed publicly.

1.2 Technologies Used

- **Networking:** VPC, subnets, route tables, Internet Gateway, NAT Gateways, Elastic IPs
- **Compute:** EC2 (Amazon Linux 2023, `t2.micro`), Launch Template, Auto Scaling Group
- **Load balancing:** Application Load Balancer, Target Group
- **Security and access:** Security Groups, IAM role, AWS Systems Manager (Session Manager)

2 Architecture

2.1 Design Summary

The VPC uses the CIDR block `10.0.0.0/16` and is divided into public and private subnets across two Availability Zones (`us-east-1a` and `us-east-1b`). The public subnets host the load balancer and the NAT Gateways. The private subnets host the web servers.

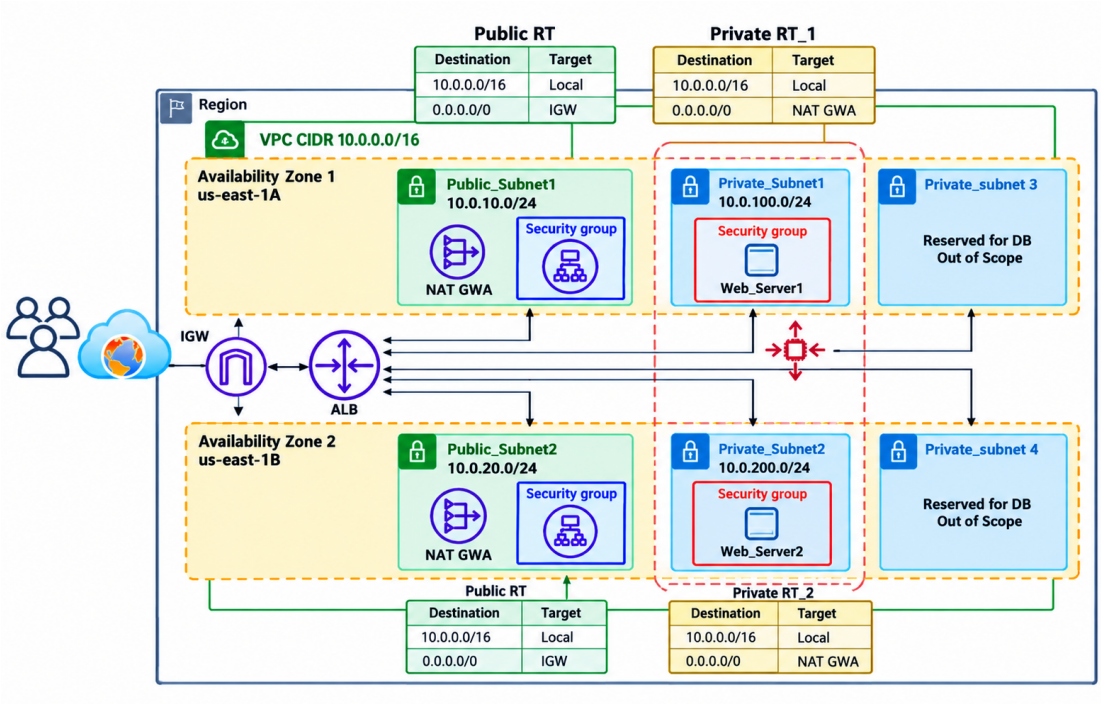


Figure 1: Architecture diagram of the deployed solution

2.2 Network Layout

Subnet	AZ	CIDR	Purpose
Public_Subnet1	us-east-1a	10.0.10.0/24	ALB node, NAT Gateway A
Public_Subnet2	us-east-1b	10.0.20.0/24	ALB node, NAT Gateway B
Private_Subnet1	us-east-1a	10.0.100.0/24	Web servers (AZ A)
Private_Subnet2	us-east-1b	10.0.200.0/24	Web servers (AZ B)

Table 1: Subnet plan

2.3 Routing

Route table	Destination	Target	Associated with
Public_RT	10.0.0.0/16	local	Both public subnets
	0.0.0.0/0	Internet Gateway	
Private_RT_1	10.0.0.0/16	local	Private_Subnet1
	0.0.0.0/0	NAT Gateway A	
Private_RT_2	10.0.0.0/16	local	Private_Subnet2
	0.0.0.0/0	NAT Gateway B	

Table 2: Route tables

Why two private route tables? Each private subnet routes through the NAT Gateway in its own Availability Zone. If one zone fails, the other zone keeps its outbound path, so the NAT layer is not a single point of failure. A single shared private route table would force both zones through one NAT Gateway.

3 Implementation

3.1 Step 1: VPC and Subnets

1. Created the custom VPC with CIDR `10.0.0.0/16` and enabled DNS hostnames.
2. Created an Internet Gateway and attached it to the VPC.
3. Created the two public and two private subnets according to the plan above.
4. Enabled auto-assign public IPv4 on the public subnets.
5. Created `Public_RT`, added a default route to the Internet Gateway, and associated it with both public subnets.

3.2 Step 2: NAT Gateways

One public NAT Gateway was created in each public subnet, each with its own Elastic IP. Then `Private_RT_1` and `Private_RT_2` were created, each with a default route to the NAT Gateway of the same Availability Zone.

3.3 Step 3: IAM Role for Systems Manager

An IAM role named `ec2tossm` was created for EC2 with the AWS managed policy `AmazonSSMManagedInstanceCo` This allows administrators to open shell sessions through Session Manager, so no SSH key pair or inbound SSH rule is needed.

3.4 Step 4: Web Servers

Two EC2 instances (`Web_Server1`, `Web_Server2`) were launched, one in each private subnet, using Amazon Linux 2023 and the `ec2tossm` instance profile, without an SSH key. The following user data installs Apache and publishes a test page:

```
#!/bin/bash
yum update -y
yum install httpd -y
systemctl start httpd
systemctl enable httpd
echo "This is server 1 in us-east-1a" > /var/www/html/index.html
```

After launch, I connected through Session Manager, confirmed that Apache was active with `systemctl status httpd`, and tested the page locally with `curl`.

3.5 Step 5: Application Load Balancer

- Created the target group `WebTG` (HTTP, port 80) and registered both instances. Health check thresholds were reduced to detect failures faster.
- Created the security group `ALBSG` allowing inbound HTTP from anywhere.
- Created the internet-facing load balancer `WebALB` across both public subnets, forwarding to `WebTG`.

Once both targets reported healthy, the ALB DNS name returned the page from the servers.

3.6 Step 6: Launch Template and Auto Scaling Group

A launch template was created with the same AMI, instance type, security group, IAM profile, and a user data script that installs Apache. An Auto Scaling Group named `ASG` was then created across both private subnets and attached to `WebTG`, with ELB health checks enabled.

Setting	Value
Minimum capacity	2
Desired capacity	4
Maximum capacity	6
Health check type	ELB

Table 3: Auto Scaling Group configuration

After the ASG launched four instances spread across both zones and they became healthy, the two manually created instances were terminated. The ASG now owns instance lifecycle and replaces any failed instance automatically.

3.7 Step 7: Security Hardening

Initially `WebSG` allowed HTTP from anywhere, which was necessary for the first tests. It was then tightened:

- `WebSG` inbound: HTTP (80) **only from** `ALBSG`.
- `ALBSG` outbound: HTTP (80) **only to** `WebSG`.

After the change, the application was still reachable through the ALB, while direct access to the instances was impossible because they have no public IPs and accept traffic only from the load balancer.

4 Verification

Test	Result
Open ALB DNS name in a browser	Page served from a private instance
Refresh repeatedly	Responses come from instances in different zones
Target group status	All targets healthy
Terminate one ASG instance	ASG launches a replacement automatically
Access via Session Manager	Works without SSH or public IP
After SG lockdown	Application still reachable through the ALB

Table 4: Verification summary

5 Design Decisions

- **Private subnets for servers:** instances have no public IP and are not directly exposed.
- **One NAT Gateway per AZ:** removes a cross-zone single point of failure, at a higher cost.
- **Session Manager instead of SSH:** no open management port, no key distribution, and access is controlled through IAM.
- **Security group chaining:** instances trust only the load balancer's security group rather than an IP range.
- **ELB health checks in the ASG:** an instance that fails the application-level check is replaced, not just one that fails at the OS level.

6 Cleanup and Cost

NAT Gateways, Elastic IPs, and the Application Load Balancer are billed while they exist, even without traffic, so all resources were removed after testing, in this order:

1. Delete the NAT Gateways
2. Delete the Auto Scaling Group
3. Release the Elastic IP addresses
4. Terminate any remaining EC2 instances
5. Delete the load balancer and target group
6. Delete the custom VPC

Billing was checked afterward to confirm that nothing was left running.

7 Lessons Learned and Future Work

- Health check settings strongly affect how quickly failures are detected and recovered.
- Per-AZ route tables are required to get real fault tolerance from NAT Gateways.
- Possible improvements: HTTPS with an ACM certificate, target-tracking scaling policies, a database tier in the reserved private subnets, VPC endpoints to reduce NAT usage, and Infrastructure as Code (Terraform or CloudFormation) for repeatable deployment.